Midterm Mastery

This report demonstrates the **circuit breaker pattern** applied to a product search service. The service uses a CPU-intensive simulateWork() function on every search request, simulating fixed cost computation. We introduced a failure mode that makes this work 100x heavier, then showed how a circuit breaker detects the degradation and automatically protects the service.

The Problem: Cascading Failure

The simulateWork() function simulates expensive per-request computation. When toggled to "heavy mode" (via /toggle-heavy), iterations increase from 1,000 to 110,000, simulating a dependency that becomes slow.

Without protection, every incoming request would be forced to wait through the heavy computation, causing response times to spike and throughput to collapse.

The Fix: Circuit Breaker Pattern

We implemented a circuit breaker with three states:

State	Behavior
CLOSED	Normal operation. Requests pass through. Consecutive slow responses (>300ms) are counted as failures.
OPEN	Tripped after 3 failures. Skips simulateWork() entirely — returns results without the expensive computation. Service stays fast.
HALF_OPEN	After 10 seconds, allows one test request. If fast → CLOSED. If slow → back to OPEN.

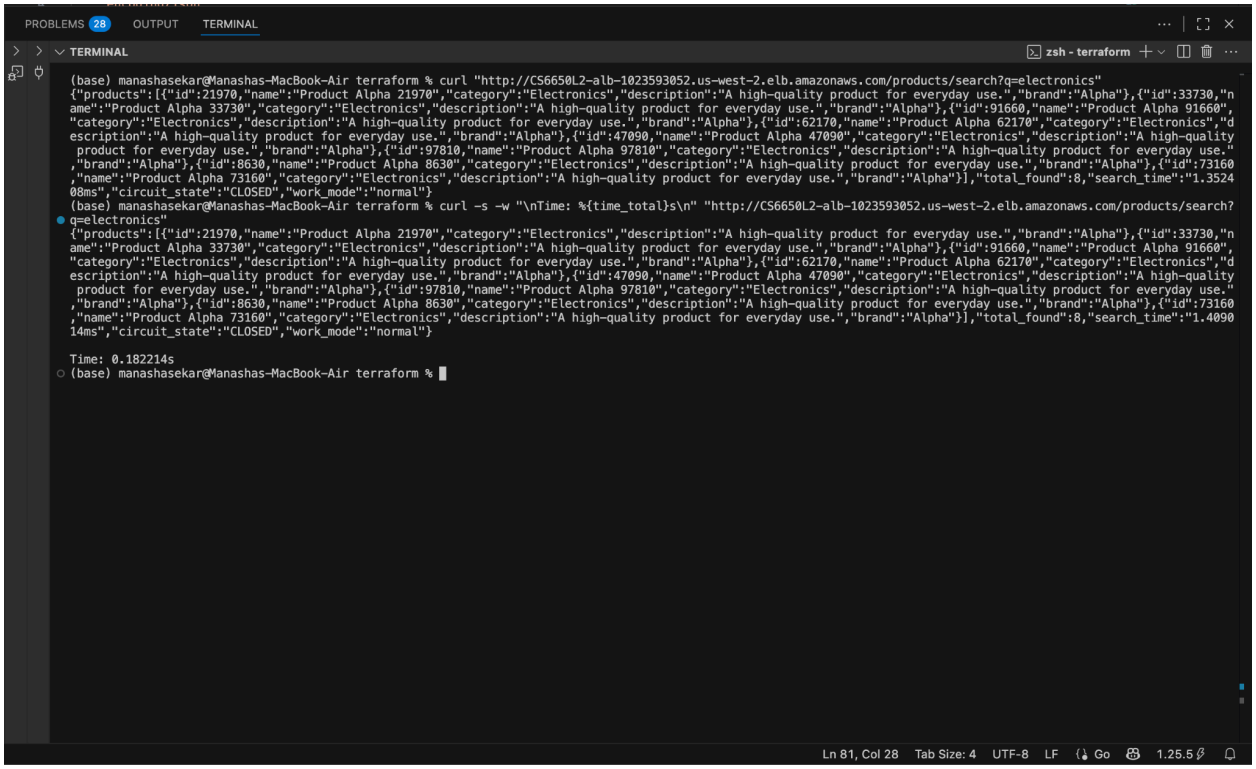
Key Configuration

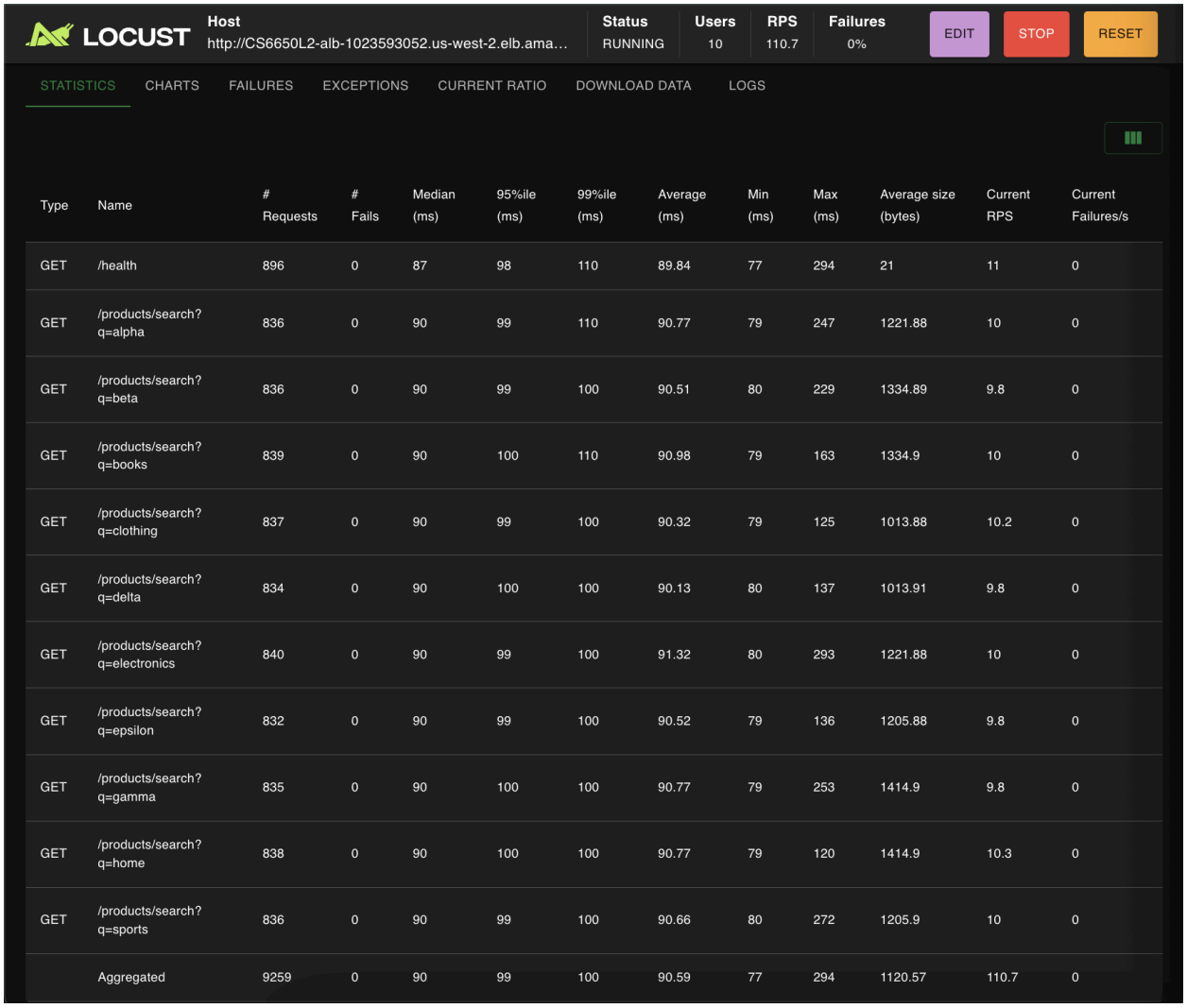
- **Failure threshold:** 3 consecutive slow responses
- **Slow response threshold:** 300ms
- **Recovery timeout:** 10 seconds

Results

Scenario 1: Normal Baseline

Metric	Value
RPS	~111
Median Response Time	90ms
99th Percentile	110ms
Failures	0%
Circuit State	CLOSED
Work Mode	normal





Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/health	896	0	87	98	110	89.84	77	294	21	11	0
GET	/products/search?q=alpha	836	0	90	99	110	90.77	79	247	1221.88	10	0
GET	/products/search?q=beta	836	0	90	99	100	90.51	80	229	1334.89	9.8	0
GET	/products/search?q=books	839	0	90	100	110	90.98	79	163	1334.9	10	0
GET	/products/search?q=clothing	837	0	90	99	100	90.32	79	125	1013.88	10.2	0
GET	/products/search?q=delta	834	0	90	100	100	90.13	80	137	1013.91	9.8	0
GET	/products/search?q=electronics	840	0	90	99	100	91.32	80	293	1221.88	10	0
GET	/products/search?q=epsilon	832	0	90	99	100	90.52	79	136	1205.88	9.8	0
GET	/products/search?q=gamma	835	0	90	100	100	90.77	79	253	1414.9	9.8	0
GET	/products/search?q=home	838	0	90	100	100	90.77	79	120	1414.9	10.3	0
GET	/products/search?q=sports	836	0	90	99	100	90.66	80	272	1205.9	10	0
	Aggregated	9259	0	90	99	100	90.59	77	294	1120.57	110.7	0



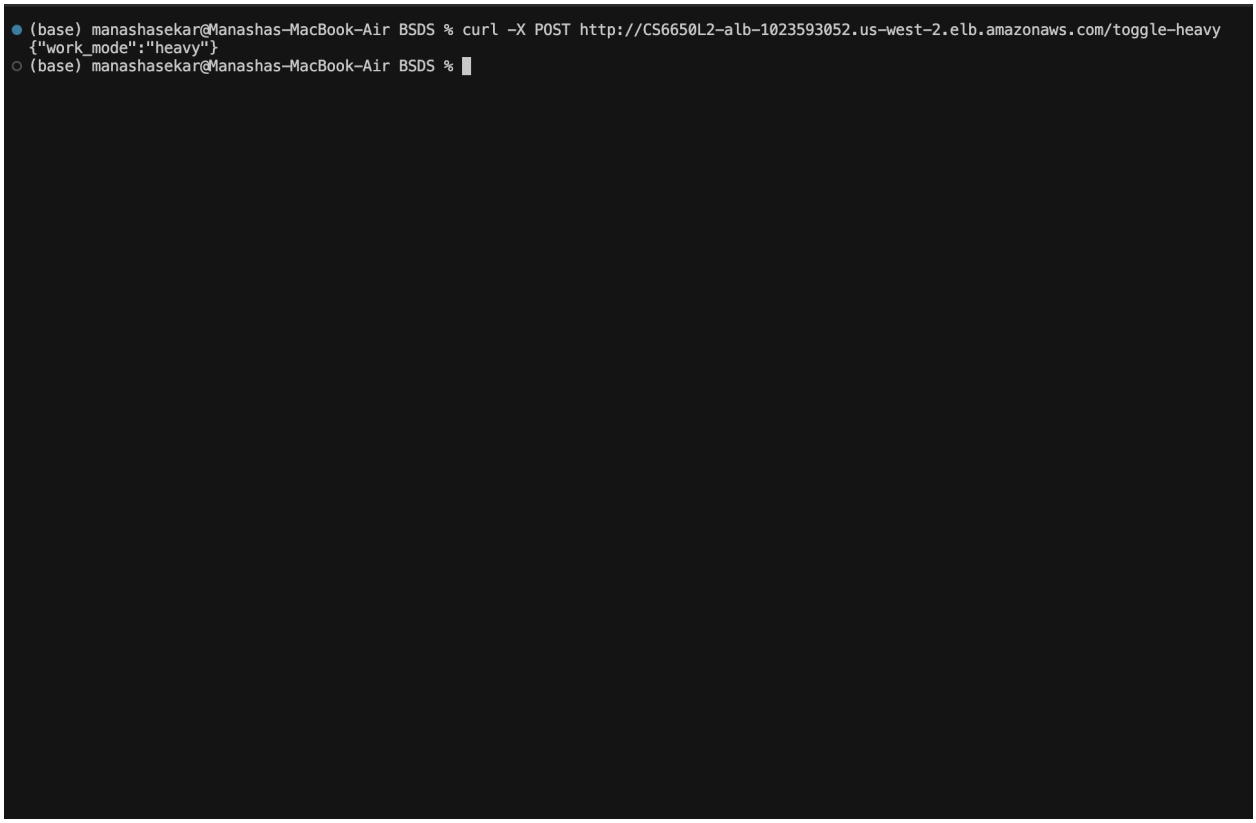
Scenario 2: Heavy Mode + Circuit Breaker

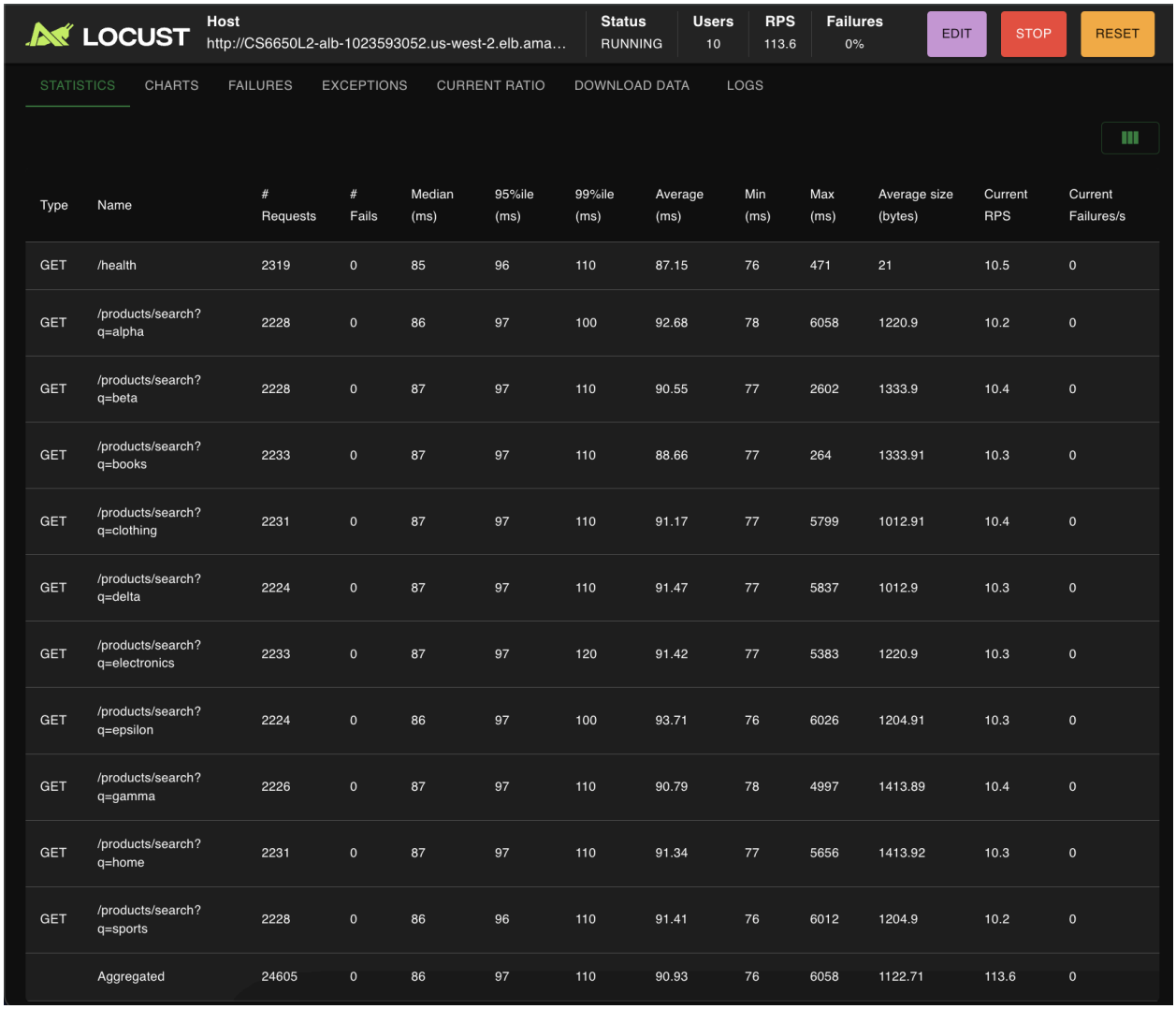
After enabling heavy mode (curl -X POST /toggle-heavy):

Phase	What Happened
First 3 requests	Heavy work runs → response times spike to **250ms (95th pctl)**, max **6,058ms**
Circuit opens	RPS drops from **~115 to ~45** during the 3 slow requests
After circuit opens	Work skipped → RPS recovers to **~115**, response times back to **~90ms**
Failures	**0%** — circuit breaker protected the service

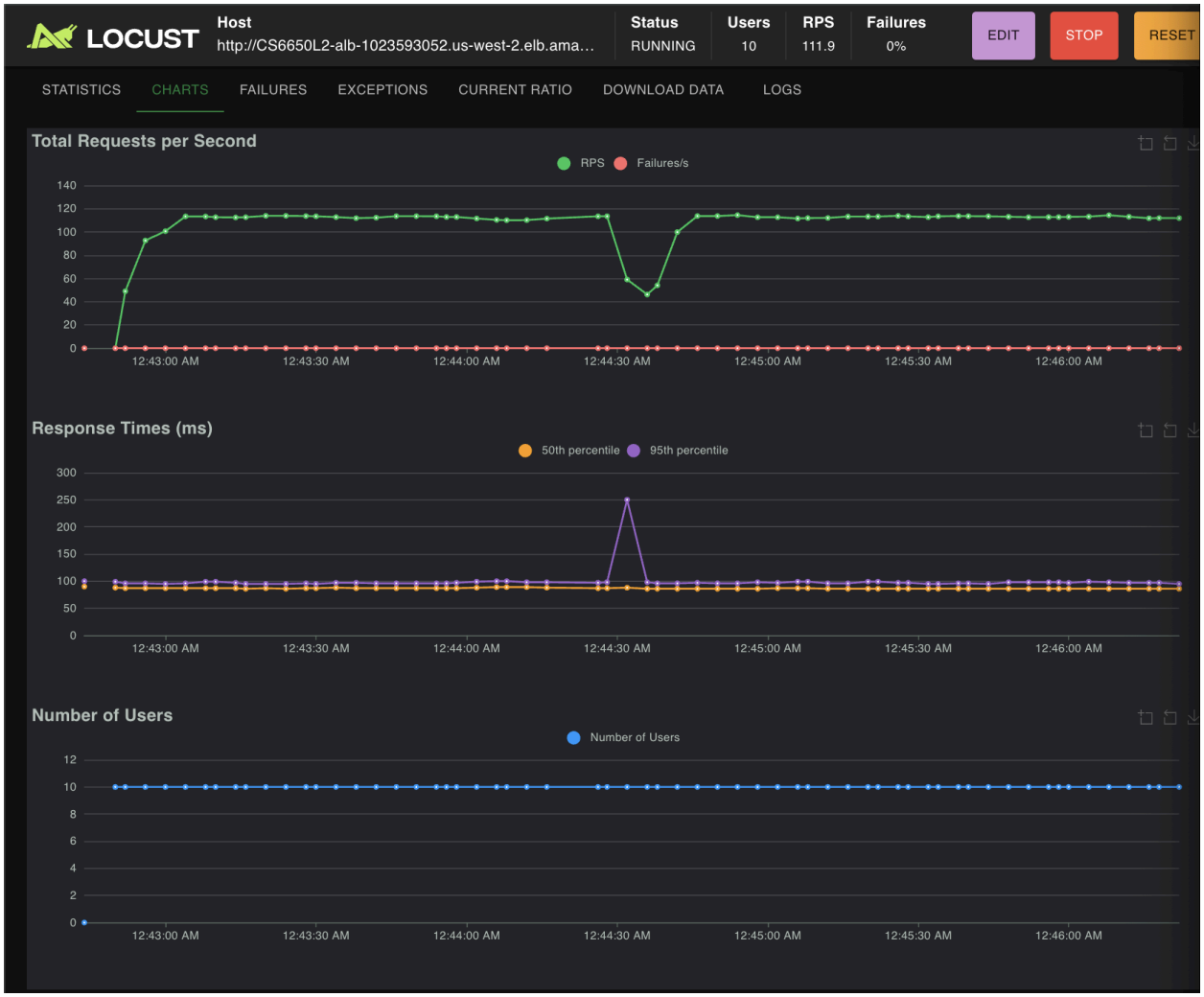
The RPS chart clearly shows:

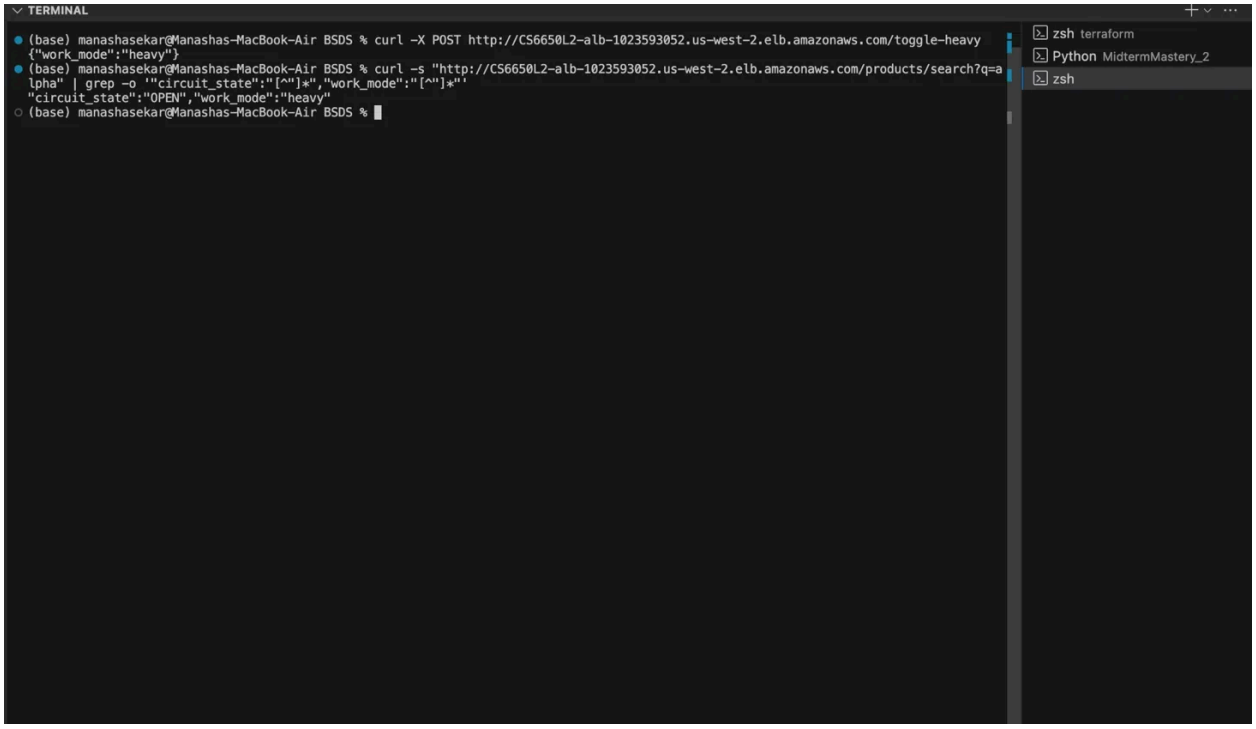
1. A sharp RPS dip to ~45 when heavy mode was enabled
2. A response time spike to 250ms (95th percentile)
3. Automatic recovery within seconds as the circuit opened





Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/health	2319	0	85	96	110	87.15	76	471	21	10.5	0
GET	/products/search?q=alpha	2228	0	86	97	100	92.68	78	6058	1220.9	10.2	0
GET	/products/search?q=beta	2228	0	87	97	110	90.55	77	2602	1333.9	10.4	0
GET	/products/search?q=books	2233	0	87	97	110	88.66	77	264	1333.91	10.3	0
GET	/products/search?q=clothing	2231	0	87	97	110	91.17	77	5799	1012.91	10.4	0
GET	/products/search?q=delta	2224	0	87	97	110	91.47	77	5837	1012.9	10.3	0
GET	/products/search?q=electronics	2233	0	87	97	120	91.42	77	5383	1220.9	10.3	0
GET	/products/search?q=epsilon	2224	0	86	97	100	93.71	76	6026	1204.91	10.3	0
GET	/products/search?q=gamma	2226	0	87	97	110	90.79	78	4997	1413.89	10.4	0
GET	/products/search?q=home	2231	0	87	97	110	91.34	77	5656	1413.92	10.3	0
GET	/products/search?q=sports	2228	0	86	96	110	91.41	76	6012	1204.9	10.2	0
	Aggregated	24605	0	86	97	110	90.93	76	6058	1122.71	113.6	0

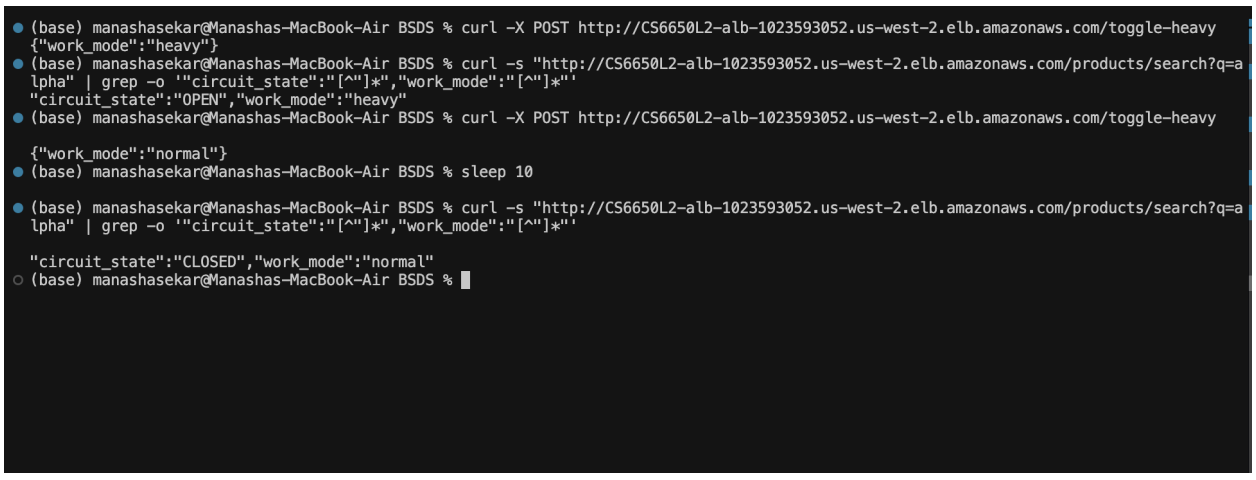

LOCUST
Host
http://CS6650L2-alb-1023593052.us-west-2.elb.ama…
Status
RUNNING
Users
10
RPS
111.9
Failures
0%
EDIT
STOP
RESET
STATISTICS
CHARTS
FAILURES
EXCEPTIONS
CURRENT RATIO
DOWNLOAD DATA
LOGS
Total Requests per Second
RPS
Failures/s
140
120
100
80
60
40
20
0
12:43:00 AM
12:43:30 AM
12:44:00 AM
12:44:30 AM
12:45:00 AM
12:45:30 AM
12:46:00 AM
Response Times (ms)
50th percentile
95th percentile
300
250
200
150
100
50
0
12:43:00 AM
12:43:30 AM
12:44:00 AM
12:44:30 AM
12:45:00 AM
12:45:30 AM
12:46:00 AM
Number of Users
Number of Users
12
10
8
6
4
2
0
12:43:00 AM
12:43:30 AM
12:44:00 AM
12:44:30 AM
12:45:00 AM
12:45:30 AM
12:46:00 AM



Scenario 3: Recovery

After toggling heavy mode off (curl -X POST /toggle-heavy):

- Waited 10 seconds for the circuit to enter HALF_OPEN
- One test request passed (fast response)
- Circuit returned to CLOSED, work mode back to normal
- Verified: "circuit_state":"CLOSED","work_mode":"normal"



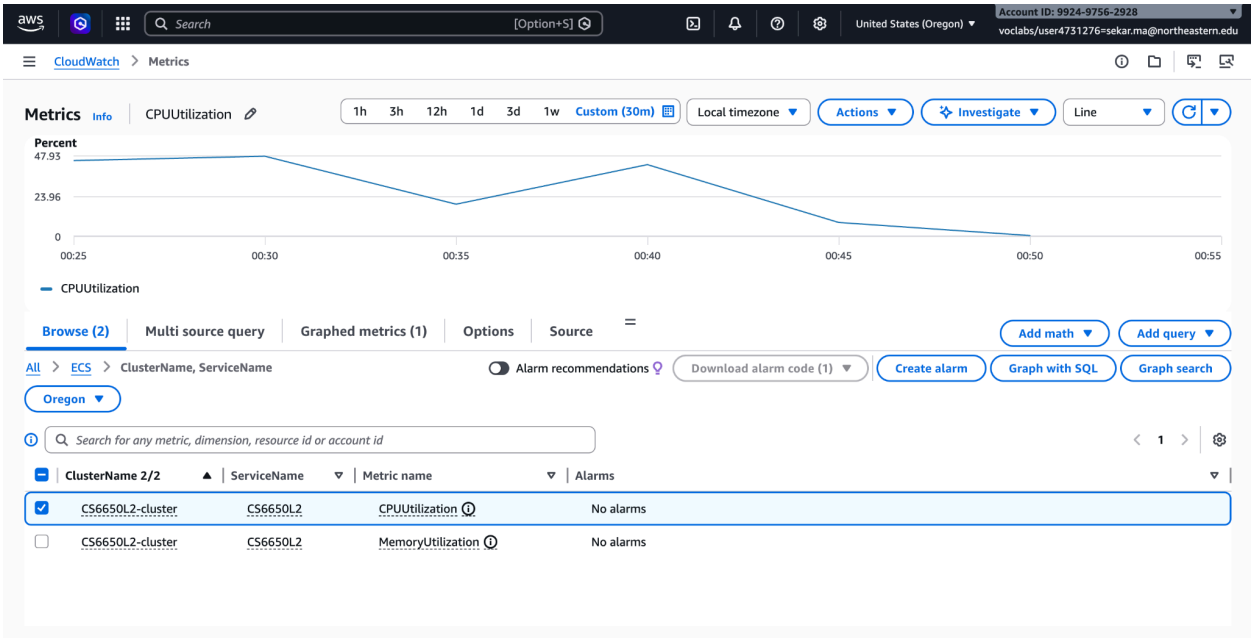
Metric	Normal	Heavy (before trip)	After Circuit Opens
RPS	~115	~45	~115 (recovered)
Median	90ms	250ms+	90ms
Max	294ms	6,058ms	294ms
Failures	0%	0%	0%

Circuit Breaker vs Other Patterns

Pattern	How It Works	When to Use
Circuit Breaker (used here)	Detects repeated failures, stops trying, self-heals	Protecting against slow/failing dependencies
Fail Fast	Rejects new requests immediately when overloaded	Preventing queue buildup under load
Bulkhead	Isolates fixed resource pools per component	Preventing one bad component from consuming all resources

We chose the circuit breaker because it is **self-healing**. it automatically detects recovery and resumes normal operation, unlike fail fast (just rejects) or bulkhead (just limits).

CloudWatch Metrics



CPU Utilization: ~48% during normal load, variation during heavy/circuit breaker cycle, then dropped after test stopped

Conclusion

The circuit breaker pattern successfully prevented a cascading failure. When the expensive computation degraded (heavy mode), only 3 requests suffered before the circuit opened and the service automatically recovered to full throughput (~110 RPS). The entire recovery happened without any manual intervention, demonstrating why this pattern is essential for building resilient distributed systems.